

TokPress: A Tokenizer-Domain Entropy Coder with a Trained Dictionary and Order-1 Fallback Cascade

Technical Report

Le Duc Minh

minh.leduc.0210@gmail.com

Abstract

TokPress is a pure-Python lossless compressor that tokenizes input with the released `o200k_base` vocabulary, applies token-level LZ77, and entropy-codes the result with rANS. All domain adaptation happens in a separate object, `TokDict`, which supplies an LZ priming buffer, a baked order-0 table, and order-1 tables for the most frequent previous-token contexts, linked by a two-level escape chain so the model itself is never sent with a record. On held-out structured logs (184 training / 46 test records), a trained `TokDict` brings the compressed/original ratio from 0.808 to **0.2565** — below `gzip -9` (0.728) and dictionary-free `zstd -19` (0.737), and within 1.29× of `zstd -19` with a matched trained dictionary (0.195). The gain is schema-specific: a dictionary trained on the wrong kind of data buys nothing. In bulk mode TokPress beats `gzip -9` on 4 of 6 corpora and `zstd -19` outright on a short-prose corpus, including an exact tie with `zstd -19` on a long-prose-novel prefix. Every reported ratio was verified by a byte-exact round-trip check.

1 Introduction

General-purpose stream compressors like `gzip` and `zstd` learn their model from the input. That cost is easy to amortize over a large file, but not over the short, independent records that structured logs, telemetry events, and per-entity metadata usually are.

`Zstandard` handles this setting with dictionary mode: a dictionary is trained offline and shared with the decoder. We ask whether a subword tokenizer trained the same way can do the equivalent job for small records. A token sequence turns multi-byte dependencies into adjacent-token ones ([Theorem A.4](#)), so a static low-order token model may capture structure that would need a higher-order byte model. We test this with the full, unfiltered `o200k_base` encoding (so byte fallback, and hence [Theorem A.2](#), always holds) and keep every trained component — the LZ priming buffer, the order-0 table, the order-1 context tables — in an external `TokDict` object rather than compiled into the codec.

The regime this targets is no longer a niche. Database engines now train per-collection dictionaries over small, schema-homogeneous documents to cut storage and I/O cost, and the IETF’s RFC 9842 (Compression Dictionary Transport, 2025) standardized `zstd/brotli` dictionary compression over HTTP, with “known-structure API responses” named as the prime fit. A shared static model over many small records is where the industry is already heading; this work is one concrete, open, end-to-end implementation of that idea, with the tokenizer taking the role of the shared dictionary.

The short version of the results is that this works in the regime it targets and not outside it. With a trained `TokDict`, small structured records compress smaller than with any dictionary-free compressor we tried, and come within 1.29× of a matched-schema `zstd -19` dictionary ([Section 6.2](#)) — a much smaller gap than the 5–7× a plain per-record encoder starts from, though we do not close it. The gain is specific to the training schema: in a cross-schema test ([Appendix B](#)), a dictionary trained on the wrong kind of record is no better than no dictionary at all. In general-purpose bulk mode, a wider entropy-table precision, separate tables for match metadata, an adaptive table that costs nothing to transmit, and a per-record PPM-style order-1 coder let TokPress beat `gzip -9` on 4 of 6 whole-file corpora and `zstd -19` outright on short prose ([Section 6.1](#)).

TokPress is open source: the implementation, wire-format documentation, and the benchmark harness are available at <https://github.com/LakoreAI/tokpress>.

Contributions.

1. A rate equation that separates the codec’s sources of redundancy, plus a coding-identity/KL treatment of tokenization ([Section 4](#), [Appendix A](#)).
2. A codec over the full released tokenizer: escape symbols so entropy tables never run out of alphabet, a 64-bit rANS state at $M = 2^{16}$ with the overflow risk checked, match metadata coded in its own tables, and a chunked-adaptive table that costs nothing to transmit ([Section 3](#)).
3. `TokDict`, an external trained dictionary (priming buffer, baked order-0, order-1 cascade), with an honest account of when it helps and when it does not, including a cross-schema test on a genuinely different record type ([Section 6](#) and [Appendix B](#)).
4. Bulk-mode measurements on six corpora — including a Wikipedia-XML dump and a long prose novel — against `gzip/bz2/lzma/zstd/brotli/lz4` and a tokenize-then-byte-compress baseline ([Section 6.1](#)).

2 Related Work

Entropy coding. TokPress uses the range variant of asymmetric numeral systems [3]. We use a static table rather than an online model because of the Krichevsky–Trofimov redundancy bound for sequentially estimated models [9]; see [Section 4.6](#).

Dictionary and LZ compression. The sliding-window LZ77 family [20] is asymptotically optimal [18], but its finite-length redundancy [12] matters at the record sizes we care

about (Section 4.4). Zstandard [16], including its `--train` dictionary mode, is our main comparison point. FemtoZip [4], which shares a dictionary across small independent documents, motivated the dictionary-priming scheme in Section 6.3. Shared static dictionaries are now a production pattern: RFC 9842 standardizes dictionary compression for HTTP, and document databases (MongoDB, Amazon DocumentDB) train per-collection dictionaries over small JSON documents. For streaming logs, LogLite [13] achieves strong compression without any pre-training by adapting to the log structure on the fly; our batch mode (Section 3) is the static-dictionary counterpart of that idea for archival, write-once workloads.

Subword tokenization. The relevant methods are BPE [5, 17], SentencePiece [10, 11], and byte-level BPE [15]. TokPress uses the released `o200k_base` piece set [14] unfiltered, and deliberately does not restrict it per domain (Section 3): a restricted piece set is only a valid tokenizer if it forms a complete hierarchical BPE merge chain, which mining a subset by longest-match over a training corpus does not guarantee.

Modeling as compression. Delétang et al. [2] treat sequence modeling and compression as the same problem. We work with a static, non-neural model meant for short records at moderate computational cost.

Vocabulary sizing. Heaps’ [8] and Zipf’s [19] laws underlie the rate–fertility trade-off of Section 4.3; Llama 3 [6] and `o200k_base` [14] are useful real-world vocabulary-size reference points.

3 System Description

The encoder tokenizes the input, applies LZ77 to the token sequence, and then entropy-codes the result. One tokenizer is used for everything; where domain adaptation happens is described below. Figure 1 shows the TokPress pipeline; the formal encode and decode procedures are given in Algorithms 1 and 2.

1. Tokenizer. The vocabulary $\mathcal{V}$ is the full, unfiltered, released `o200k_base` encoding ($|\mathcal{V}| \approx 2 \times 10^5$), used exactly as published via its byte-exact `_encode_bytes/decode_bytes` pair, so arbitrary binary input (not only valid UTF-8) round-trips exactly. Byte-level fallback ($\mathcal{B} \subseteq \mathcal{V}$) is guaranteed by `o200k_base` itself, so Theorem A.2 applies unconditionally. We never restrict the vocabulary by domain, because a restricted vocabulary is only valid if it forms a complete BPE merge chain, and mining pieces by longest-match does not guarantee that. A domain vocabulary trained by the project’s `train-vocab` tool can be substituted; the codec is tokenizer-agnostic.

2. Token-level LZ77. A greedy hash-based parser works on token IDs. History is optionally primed with a *priming buffer* from a trained TokDict (Section 3; up to 8192 tokens by default, a trainable parameter), so matches can refer to material learned from other records. Without a TokDict, history is empty and every record is compressed on its own.

A match of length l costs four tokens, so matches are emitted only when $l \geq 3$ (Remark 4.4). The match-flag symbol is the tokenizer’s own vocabulary size, so it never collides with a real token id.

3. Entropy coding. A scalar rANS coder runs at precision $M = 2^{16} = 65536$ (table-log 16; Appendix C explains why this needs a 64-bit coder state). The encoder builds whichever of eight modes apply and keeps the smallest (Algorithm 1):

- **raw:** fixed-width bit-packed tokens, the universal fallback when nothing else applies or wins;
- **sparse order-0:** a per-record table over the record’s own distinct symbols, with a reserved *escape symbol* for anything beyond the $M - 1$ most frequent — without this, a record with more than M distinct symbols (ordinary prose can, at a small M) would have no valid table and would silently fall back to raw bit-packing;
- **split:** LZ match metadata (the distance-high, distance-low, and length bytes of a match tuple) is coded in its own three small tables, plus a role-bit table distinguishing literals from match positions, all separate from the literal-token table. Match-metadata bytes are near-uniform over $[0, 255]$, a completely different distribution from the literal tokens’ Zipfian one; mixing them in one table dilutes both. Measured directly, separation saves 6–16% of the entropy-coded payload;
- **chunked adaptive order-0:** no table is transmitted at all. Each chunk after the first is coded against a Laplace-smoothed count of the chunks that came before it, so the decoder derives the identical table from what it has already decoded (Section 4.6 explains why we did not attempt fully per-symbol adaptive coding);
- **adaptive split:** the previous two combined — match metadata stays in static small tables while the literal sub-stream gets the chunked-adaptive treatment. The two savings are largely independent, so a record can win from either or both;
- **TokDict order-0/order-1 cascade** (Section 3): the only mode with pre-trained, cross-record state, used only when a TokDict is supplied;
- **per-record PPM order-1:** an adaptive order-1 cascade over the whole token stream — the previous token’s context table is tried first, an escape from it falls through to a Laplace-smoothed order-0 table that always covers the symbol, and each context reserves a count-based (PPMC-style) escape share, so a low-support context mostly falls through and never hurts;
- **PPM split:** the same order-1 cascade applied to the literal sub-stream, with match metadata in its own tables (the two wins stack).

(An empty-input fallback handles the zero-byte edge case.)

TokDict: an external, explicitly-trained dictionary. All trained, domain-adapted state lives in an external, opt-in object, trained by the caller on a sample of representative records (Algorithm 3), saved to a portable file, and supplied

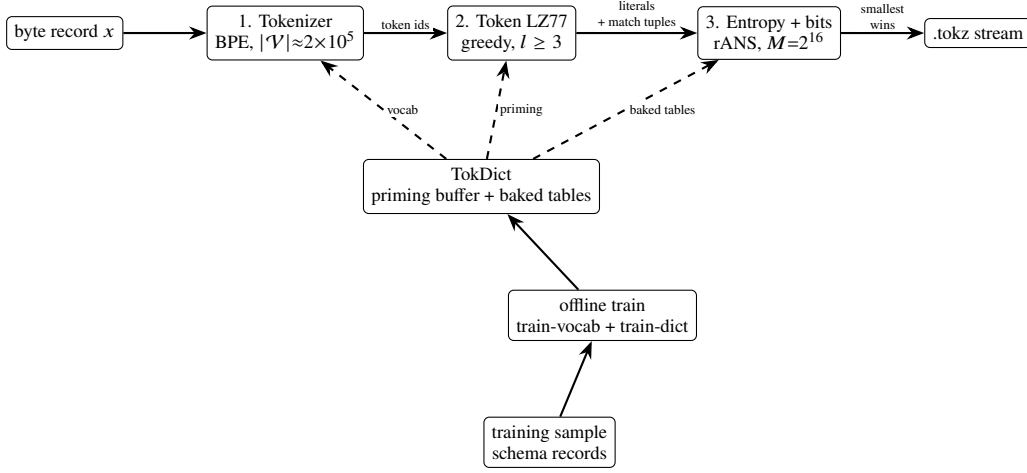


Figure 1: The TokPress pipeline: tokenize $\rightarrow$ token-level LZ77 $\rightarrow$ entropy-coded bitstream, with the trained TokDict supplying the vocabulary, an LZ priming buffer (so matches can reach into other records), and baked order-0/order-1 tables.

Algorithm 1 TokPress encode

Require: input bytes x ; optional TokDict Θ
Ensure: a single-record TokPress stream

- 1: $t \leftarrow \text{TOKENIZE}(x)$ $\triangleright$ byte-exact, arbitrary binary
- 2: $s \leftarrow \text{LZ77}(t)$ $\triangleright$ greedy; primed with Θ 's buffer if given
- 3: $C \leftarrow \{\text{RAW}(s), \text{SPARSE}(s), \text{SPLIT}(s)\}$
- 4: $C \leftarrow C \cup \{\text{ADAPTIVE}(s), \text{ADAPTIVESPLIT}(s)\}$
- 5: **if** a dictionary Θ is given **then**
- 6: add $\text{DictCASCADe}(s, \Theta)$ to C $\triangleright$ see Algorithm 4
- 7: **end if**
- 8: **return** $\arg \min_{c \in C} |c|$ $\triangleright$ every candidate is built; the smallest wins

Algorithm 2 TokPress decode

Require: a TokPress stream; optional TokDict Θ
Ensure: the original bytes x

- 1: parse the magic, version, mode, and payload header
- 2: decode the LZ-token stream with the mode's rANS tables $\triangleright$ escape lists resolved in order
- 3: $t \leftarrow \text{LZDECOMPRESS}(s, \Theta.\text{priming})$
- 4: **return** $\text{DETOKENIZE}(t)$

explicitly at compress/decompress time. Training accumulates: (i) a priming buffer (the sample records' own token streams, concatenated up to a cap); (ii) order-0 counts over the resulting LZ-token stream; and (iii) order-1 counts for every (previous-token, next-token) pair observed. The order-0 and every order-1 table use the same escape-capping construction as the sparse mode, but not the same escape share: order-0 reserves a small fixed 1.5%, while each context table reserves a PPMC-style count-based share estimated from its own counts, because a context table sees far fewer observations than order-0 does, so "this specific transition was not in training" is common for it, not rare (Section 6.4 has the ablation). Only the top 64 contexts with at least 20 observed transitions get their own table; a low-support context is simply never given one, at no cost. To encode a symbol whose previous symbol has a trained context table, we try that table first; an escape falls through to the order-0 table; an escape from *that* carries the symbol's value out-of-band as an explicit

Algorithm 3 TokDict training

Require: sample records $\{r_1, \dots, r_n\}$; tokenizer; cap P on priming tokens
Ensure: a TokDict $\Theta = (\text{priming}, \text{order0}, \text{contexts})$

- 1: $t_i \leftarrow \text{TOKENIZE}(r_i)$ for each i
- 2: $\text{priming} \leftarrow$ concatenation of the t_i , capped at P
- 3: $s_i \leftarrow \text{LZ77}(t_i)$ for each i
- 4: accumulate order-0 counts over all s_i ; and order-1 counts for every adjacent pair
- 5: bake the order-0 table, capped to $M-1$ symbols plus one escape (share 1.5%)
- 6: keep the top-64 contexts with ≥ 20 transitions; bake each with a PPMC-adaptive escape share
- 7: compute the 8-byte fingerprint; **return** Θ

integer (Algorithm 4). Which table to try depends only on the *previous* symbol and the dictionary's fixed metadata — never on the current symbol's value — so the decoder can derive the same cascade from what it has already decoded, without transmitting anything about the table choice.

Correctness. The cascades were easy to get wrong, and three real bugs were found and fixed only by testing many varied payloads rather than one fixed one. First, rANS encodes in reverse logical order (Section 4.6), so a position needing a two-event cascade (e.g. context-then-order-0 fallback) must have its *encode calls* issued in the *opposite* micro-order from how the decoder consumes them; getting this backwards corrupts output only on inputs that exercise the escape path. Second, in adaptive-split mode the escape list is built in a forward pass *first* (unlike every other mode, where it is built during the reverse loop), so reversing it a second time at the end silently reordered escaped values — again, only when a record had more than one order-0 escape. Third, a table with exactly one active symbol has frequency exactly M (certainty), which needs $\log_2 M + 1$ bits, one more than the field width used for every other frequency; transmitting it directly truncated 2^{16} to 0 on the wire. Transmitting frequency -1 instead (range $[0, M-1]$) fixes this uniformly. All three were caught by tests that loop over many payload shapes, random synthetic symbol streams, or extreme record sizes.

Algorithm 4 TokDict order-1 $\rightarrow$ order-0 $\rightarrow$ escape cascade (encode one symbol)

Require: symbol y ; previous symbol x ; TokDict Θ

- 1: $ctx \leftarrow$ the trained context table for x , if one exists
- 2: **if** y is in ctx **then**
- 3: rANS-encode y against ctx $\triangleright$ no fallback needed
- 4: **else if** y is in the order-0 table **then**
- 5: rANS-encode y against the order-0 table; then a ctx -escape if ctx exists
- 6: **else**
- 7: append y to the out-of-band escape list
- 8: rANS-encode the order-0 escape; then a ctx -escape if ctx exists
- 9: **end if**
- 10: $\triangleright$ encode calls are issued in reverse logical order — see [Section 3](#)

4 Theoretical Foundation

Let $\mathcal{B} = \{0, \dots, 255\}$ be the byte alphabet; a message is $x \in \mathcal{B}^*$ with $|x| = n$. We write H for entropy, I for mutual information, and $D(p \parallel q) = \sum_x p(x) \log_2(p(x)/q(x))$ for KL divergence.

The coding identity — the expected length under a mismatched model is the source entropy plus a KL-divergence penalty, $\mathbb{E}_p[-\log_2 q(X)] = H(p) + D(p \parallel q)$ — is stated and proved in [Appendix A \(Lemma A.1\)](#); we use it throughout. All formal statements below are collected in [Appendix A](#) and only the claims are quoted here.

4.1 Entropy invariance under tokenization

Definition 4.1 (Greedy longest-match tokenizer). Fix $\mathcal{V}$ with $\mathcal{B} \subseteq \mathcal{V}$. The map $T : \mathcal{B}^* \rightarrow \mathcal{V}^*$ emits, at each position, the longest piece of $\mathcal{V}$ that prefixes the remaining suffix, and advances by its length. The concatenation map $\kappa : \mathcal{V}^* \rightarrow \mathcal{B}^*$ recovers the bytes.

Tokenization is invertible and preserves source entropy ([Theorem A.2](#)): byte fallback guarantees the greedy matcher never stalls, so $\kappa(T(x)) = x$, T is injective, and $H(T(X)) = H(X)$. A direct consequence is that tokenization cannot reduce the information content of a message — every compression gain must come from *modeling*, never from the transform ([Corollary A.3](#)).

The bulk-mode results in [Section 6.1](#) are consistent with this: without a domain-trained model, TokPress is a general LZ and entropy-coding pipeline and gets no advantage from tokenization alone.

4.2 Gain from token context

Let $\bar{\ell} = |x|/m$ be the mean bytes per token, and $R_0^T = H(T_1)/\bar{\ell}$, $R_1^T = H(T_1 \mid T_0)/\bar{\ell}$ the per-byte rates of order-0 and order-1 token models. The per-byte difference between the two is the adjacent-token mutual information, $R_0^T - R_1^T = I(T_0; T_1)/\bar{\ell} \geq 0$ ([Theorem A.4](#)).

Because a token spans an average of $\bar{\ell}$ bytes, the previous token carries several bytes of context. For sources with multi-byte dependence, a first-order token model may therefore hold more useful context than a first-order byte model. How big

the effect is depends on the source and tokenizer; [Section 6.4](#) measures it on held-out data via $I(T_0; T_1)$.

Remark 4.2 (State-space reduction). A byte-level order- m model needs $256^m = 2^{8m}$ conditioning contexts; a token-level order-1 model needs $K = |\mathcal{V}|$. At comparable context reach the ratio is $2^{8\bar{\ell}}/K$, i.e. $2^{32}/2^{16} = 2^{16}$ at $\bar{\ell} = 4$, $K = 2^{16}$. This is a statement about parameter count, not expressivity: token boundaries vary with the input, and correlations inside a token are represented by the vocabulary rather than by the context model.

4.3 Vocabulary sizing

Let $K = |\mathcal{V}|$ and let $r(K) = a + \beta \ln K$ be the mean fertility (Heaps-type growth, with $a \geq 1$ forced by byte fallback), and suppose token ranks follow a Zipf law with exponent α . Under that Zipf law the vocabulary entropy grows like $\frac{1}{2} \log_2 K$ at $\alpha = 1$ ([Lemma A.5](#)), and the per-byte rate is monotone in $\log K$, so there is no interior optimum — only a knee ([Proposition A.6](#)).

For $\alpha \approx 1$, $H(K) \approx \frac{1}{2} \log_2 K + c$. Tokenizers with $K \approx 50$ –128k commonly yield 9.5–10.5 bits/token, which matches $O(\frac{1}{2} \log_2 K)$ growth; a $\log_2(\ln K)$ model would predict only about 3.5 bits at $K = 2^{16}$.

The useful operating point is a knee, not an interior minimum. Once the corpus’s useful subword inventory is covered, extra entries add little fertility while table cost keeps growing with K . That argues for capping the *tokenizer*’s vocabulary at a domain-appropriate K ; TokPress instead uses the full released o200k.base vocabulary ([Section 3](#)) and applies the cap at the *entropy-table* layer instead — $M-1$ symbols per baked table with an escape for the rest ([Appendix C](#)) — since a real, valid tokenizer vocabulary cannot be truncated post hoc. We did not fit a , β , or α to locate the knee for $\mathcal{V}$ ’s real $K \approx 2 \times 10^5$ ([Section 7](#)).

4.4 Token-level LZ77 and the small-record crossover

Definition 4.3 (Match and greedy parse). A match at position i with distance $d \leq D$ and length $l \leq L$ requires $t_{i+k} = t_{i-d+k}$ for $k < l$. The parser replaces l tokens by the 4-token tuple (FLAG, $\lfloor d/2^8 \rfloor$, $d \bmod 2^8$, l), with FLAG = $|\mathcal{V}|$, escaping a literal FLAG as (FLAG, 0, 0, 0). The implementation uses $D = 2^{15}$, $L = 255$.

Remark 4.4 (Match-length threshold). A match of length l costs 4 *tokens*, so raw *token count* only decreases for $l \geq 5$. But we minimize entropy-coded *bits*, not token count, and once match metadata is coded separately in cheap, concentrated tables ([Section 3](#)), a match’s 4-token overhead can cost far fewer bits than the $l-1$ literal tokens it replaces, even at $l = 3$. Measured on every corpus we tried, $l \geq 3$ compressed 3–7% smaller than $l \geq 5$ under the current architecture; under a plain fixed-width bit-packed or single-shared-table encoding the reverse held ([Section 6.3](#)), which is why an earlier revision moved the threshold up to 5 and this one moved it back down.

Sliding-window LZ77’s rate carries a finite-length redundancy of $\Theta(\log \log N / \log N)$ bits/byte over N bytes ([Theorem A.7](#)). Representative asymptotic estimates are roughly 0.3–0.5 bits/byte at $N = 200$ B, 0.1 bits/byte at $N = 1$ MB,

Table 1: Relation between codec design choices and the terms in Theorem A.10.

Lever	Term	Mechanism
Trained TokDict	$D(\tilde{p} \parallel \tilde{q})$	concentrates long-range dependence into adjacent-token MI
Order-1 modeling	$H(\tilde{p})$	recovers $I(T_0; T_1)/\bar{\ell}$
Larger M	$\varepsilon_{\text{quant}}$	finer frequency grid
Baked (shared) model	$\varepsilon_{\text{header}}$	no per-record table on the wire
Static, no adaptation	R_{model}	removes KT learning cost
Few lanes / batching	$\varepsilon_{\text{flush}}$	fewer flushed states
Threshold $l \geq 3$	ε_{LZ}	no expand-by-one matches

and 0.05 bits/byte at $N = 1$ GB. The finite-length term is therefore relevant for the short records we target.

Remark 4.5 (The crossover). A pretrained static model has no within-record learning term, so its rate is roughly independent of N . A crossover N^* is expected where

$$\frac{\hat{H}}{r} + \varepsilon_{\text{coder}} + \frac{R_{\text{header}}}{N} < h + c \frac{\log \log N}{\log N},$$

Our experiments cover records of about 200–540 B and bulk-file inputs. A size sweep over per-record compression (Section 6.6) locates N^* for one schema-homogeneous record family at roughly 1–2 KB, where per-record adaptive zstd overtakes the trained static model.

4.5 rANS and the master equation

Definition 4.6 (rANS step). With frequencies $f_i > 0$, total $M = \sum_i f_i$, cumulative c_i , and $q_i = f_i/M$: encoding symbol i from state s gives $s' = \lfloor s/f_i \rfloor M + (s \bmod f_i) + c_i$; decoding sets $b = s' \bmod M$, recovers the unique i with $c_i \leq b < c_i + f_i$, and returns $s = f_i \lfloor s'/M \rfloor + (b - c_i)$. States are renormalized into $[L, 2^{16}L)$ with $L = 16M$ (this ratio, not the absolute value of L , is what the implementation holds fixed as M changes — Appendix C), emitting or consuming 16-bit words.

The rANS step is a bijection and decodes uniquely given the exact frequency model (Theorem A.8), and its expected per-symbol length under a mismatched model q against the true p is $\sum_i p_i \log_2(M/f_i) = H(p) + D(p \parallel q)$ (Theorem A.9). So model changes show up directly in the rate: vocabulary selection and conditional modeling affect the mismatch term, while finite precision adds a coder overhead.

Remark 4.7 (Block flush cost). An interleaved coder flushing W states of 32 bits per block of M_b tokens pays $\varepsilon_{\text{flush}} = 32W/M_b$ bits/token. At $W = 8$, $M_b = 50$ (a single log line) this is 5.12 bits/token — larger than the entire model term. For short blocks the cost falls if fewer lanes are used or several records are coded in one block so that $M_b \gg 32W$.

The master rate equation, which separates the codec’s redundancy sources, is stated and proved in Appendix A (Theorem A.10); its terms map to design choices in Table 1.

By Theorem A.2, tokenization does not change the source entropy. Its role is to change how much model mismatch is attainable at a fixed context order.

4.6 Static versus adaptive models

An online-adaptive frequency model pays a Krichevsky–Trofimov redundancy [9] of roughly

$$R_{\text{model}} \approx \frac{C(K-1)}{2} \log_2 \frac{M}{C} \text{ bits}$$

over C context classes, K symbols, and M tokens. At $C = 256$ and $K = 2^{16}$, that is about 8.4×10^6 parameters, and the redundancy drops below ε bits/token only when $M/\log_2(M/C) \gg 10^8$ — far beyond the roughly 50 tokens in a 200 B record. So TokPress bakes its TokDict tables statically rather than adapting them per-symbol online: training cost is shared across the corpus, and model error shows up in the fixed $D(p \parallel q)$ term instead of an online learning cost. Table 3 shows the measured progression from no dictionary to a fully trained order-0/order-1 TokDict. Inside a single record with no shared training data, the same argument runs the other way: Section 6.4 reports that a per-record static order-1 table, with no corpus to amortize its transmission cost, consistently loses to plain order-0 — the opposite of the corpus-amortized case, and the reason TokDict’s order-1 tables are trained once across many records rather than fit fresh per record. The chunked-adaptive mode (Section 3) is a smaller, safer step in the online direction: it never transmits a table, only adapts an order-0 model, and only within records long enough to have several chunks of history.

Formally, over N_{tot} corpus bytes and per-record token count M_{rec} ,

$$b = \frac{\hat{H}}{r} + \varepsilon_{\text{quant}} + \varepsilon_{\text{flush}} + \frac{R_{\text{vocab}} + R_{\text{tables}}}{N_{\text{tot}}} + \frac{R_{\text{rec}}}{M_{\text{rec}}},$$

which specializes Theorem A.10 to independently coded short records. The shared-model terms amortize over the corpus; the record header is paid per input.

5 Experimental Setup

Corpora. (i) A real structured-log corpus, 230 records (≈ 200 –540 B each), used for both the target-regime progression (Section 6.2) and TokDict training, split 184 training / 46 held-out (80/20). (ii) A second, smaller (50-record) sample of the same log schema, split 35/15, as a lower-training-data comparison. (iii) A Python source file (≈ 8000 lines), split into 145 function/class-level snippets by top-level `def/class` boundaries and filtered to the 100–2000 B range (108 records), used for the cross-schema experiment (Appendix B) — a genuinely different schema from the JSON records, not just a different JSON shape. (iv) Six whole-file corpora for bulk measurement (Section 6.1): a short English-prose file and a C-source file from the Canterbury corpus, the Python source file as one blob, the JSON log file as one blob, a 2 MB prefix of a Wikipedia-XML dump, and a 1 MB prefix of a long prose novel. Only the long-text prefixes are used: the rANS stage is pure Python and does not scale to the full files in reasonable time. No compressor is trained on any bulk-mode corpus.

Competitors. `gzip -9`, `bz2 -9`, `lzma -9e`, `zstd -19` (with and without a trained dictionary, per experiment), `brotli -11`, `lz4`, and a “parmar-style” baseline (tokenize with `o200k.base`, pack token ids as fixed-width 3-byte integers, pipe through `lzma -9e`) standing in for tokenize-then-byte-compress pipelines that do not entropy-code tokens directly.

Correctness checks. Every reported ratio comes with a byte-exact round-trip check on the same data, verified in-line in the benchmark harness. The test suite (99 tests) covers bitstream and rANS round-trips (including the single-active-symbol edge case behind the frequency-truncation bug), token-level LZ77 round-trip, byte-exact tokenizer round-trip on arbitrary binary (including invalid UTF-8), full codec round-trips across payload shapes (JSON-like, code-like, empty, all 256 byte values, multi-byte UTF-8, high-entropy random data, a long run of one repeated byte), TokDict training/save/load/escape-cascade round-trips over many varied held-out records (the regression test that caught the escape-cascade ordering bug), the batch and indexed-batch modes, the vocabulary trainer (merge-chain validity, tiktoken agreement), and black-box package/CLI tests.

Throughput. All measurements are in-process, with encoder and decoder constructed once and reused; Table 4 reports means and standard deviations over 3 runs per corpus.

6 Results

All ratios are compressed size divided by original size. We separate general-purpose bulk compression with no trained dictionary (Section 6.1) from domain-trained small-record compression with a TokDict (Section 6.2).

6.1 General-purpose bulk mode

No compressor is trained on any corpus in this subsection; every backend runs dictionary-free.

TokPress (TP) beats `gzip -9` on 4 of the 6 corpora — prose, both long-text corpora, and the Python source — and beats `zstd -19` outright on the short-prose corpus (0.298 vs. 0.324), including an exact tie on the prose-novel prefix. It loses to `gzip` on the other two, both of which already have small enough vocabularies that the escape/table-transmission cost this system’s optimizations target was never the bottleneck (Section 7). It never beats `bz2` or `lzma`; `brotli -11` is the strongest baseline on most corpora. As Corollary A.3 predicts, tokenization alone does not reduce source entropy — these wins come from the entropy-coding, LZ-metadata-separation, and per-record order-1 choices in Section 3, not from tokenization itself.

6.2 Target regime: ratio progression

This experiment uses the real structured-log corpus, 230 records of about 200–540 B, split 184 training / 46 held-out (80/20). Records are compressed independently, with no cross-record state beyond what a trained TokDict supplies.

On the smaller 50-record log sample (35/15 split), the same progression is $0.793 \rightarrow 0.574 \rightarrow 0.572$: order-1 barely

moves the ratio at this training scale, because only 4 contexts meet the 20-transition threshold with this little data (vs. 64 at the larger scale).

With a trained TokDict, TokPress beats every dictionary-free competitor (`gzip`, `zstd`, `lz4`) by a wide margin and comes within $1.29\times$ of `zstd -19` given the same training records — down from the $5\text{--}7\times$ gap a per-record encoder with no dictionary starts from. It does not close that gap: `zstd`’s COVER/FastCover dictionary training and FSE entropy tables are more mature than our concatenation-based priming buffer and order-0/order-1 tables. These ratios are stable, not a single-split artifact: over three seeded 80/20 shuffles of the same corpus, `tokpress+dict` is 0.2630 ± 0.0007 , `tokpress+dict+batch` 0.2327 ± 0.0048 , and `zstd -19` with a matched dictionary 0.1973 ± 0.0003 (per-record comparison; the gap to `zstd`’s dictionary is consistent across splits).

Attributing the batch win. Compressing all held-out records as one adaptive stream (`compress_many`) reaches 0.2286, but the mechanism depends on whether a dictionary is present. Without one, the entire win is the cross-record adaptive entropy model: per-record adaptive coding is 0.808, the one-stream batch 0.262. With a trained TokDict, the adaptive model adds nothing — forcing the batch onto the dictionary’s static baked table already yields 0.2286, byte-identical to the mode-selection result — so the whole step from per-record (0.2565) is shared LZ history (records matching each other inside the single concatenated stream) plus framing: the 46 ten-byte TOKZ headers collapse into one 101-byte TOKB container. Batch mode is therefore two different wins depending on regime: an adaptive-model win when no dictionary is trained, and an LZ-history-and-amortization win when one is.

6.3 Dictionary priming

Table 3 decomposes the dictionary’s gain into its three layers, measured with each layer toggled independently via the training harness (the rows below “no dictionary” are all the TokDict candidate in isolation, so no other mode masks the dictionary’s own size). The baked order-0 table is the largest single step ($0.808 \rightarrow 0.425$): it replaces a per-record frequency table with a pre-trained shared one at zero per-record cost. Priming the token-level matcher with a shared buffer (up to 8192 tokens; Appendix C) drawn from TokDict’s own training records is the next step ($0.425 \rightarrow 0.288$): most of it is priming-buffer LZ matches reaching into other records. Order-1 conditioning is the smallest layer ($0.288 \rightarrow 0.2565$), evaluated separately below.

Match-metadata separation: negative result that became positive. Giving LZ match-metadata bytes their own rANS tables, separate from literal tokens, was tried in an earlier iteration at $M = 4096$ and found to *hurt*: splitting a 4096-slot precision budget across more tables costs the far more frequent literal tokens more than it helps rare match metadata. At $M = 2^{16}$ each table gets its own full budget instead of sharing one, and the same idea now helps substantially — measured directly, separation saves 6–16% of the entropy-coded payload (Section 3’s split mode). The win is a direct

Table 2: Bulk mode, whole-file corpora, no trained dictionary. All 6 files round-tripped losslessly for TokPress and parmar-style.

Corpus	TP	gzip	bz2	lzma	zstd	brotli	parmar-style
prose (152 KB)	0.298	0.356	0.284	0.319	0.324	0.306	0.318
C code (11 KB)	0.338	0.280	0.273	0.272	0.271	0.244	0.291
Python code (300 KB)	0.233	0.242	0.217	0.214	0.219	0.206	0.220
JSON logs (122 KB)	0.195	0.158	0.132	0.136	0.140	0.133	0.192
Wikipedia XML (2 MB)	0.333	0.361	0.288	0.286	0.295	0.280	0.292
prose novel (1 MB)	0.298	0.363	0.266	0.295	0.298	0.291	0.293

Table 3: Target-regime progression on the structured-log corpus (184/46 split), and the competitor baselines measured on the same 46 held-out records.

Stage / competitor	Ratio
No dictionary (TokPress, best per-record mode)	0.808
+ TokDict order-0 table (no priming)	0.425
+ LZ priming buffer	0.288
+ order-1 context tables	0.2565
gzip -9	0.728
zstd -19, no dictionary	0.737
lz4	1.001
zstd -19, matched trained dictionary	0.195

consequence of the table precision change, not a change to the idea itself.

6.4 Order-1 conditioning

[Theorem A.4](#) motivates order-1 conditioning, but realizing a genuine gain from it required fixing two things found only by measuring on real data.

Escape-share ablation. TokDict’s order-1 context tables reuse the order-0 table’s escape-capping construction, but their escape share is not the same constant. A context table sees far fewer observations than order-0 overall, so “this specific transition wasn’t in training” is common for it, not rare: holding the share at order-0’s 1.5% made order-1 conditioning *worse* than order-0-only (0.351 vs. 0.346 on the structured-log corpus, 184/46). We therefore estimate the share per context from its own counts — a PPMC-style count-based escape probability (escape mass equal to the number of distinct next-symbols seen in that context) — which yields 0.288 → **0.2565** ([Table 3](#)). A parameter sweep over a fixed context share (order-0 held at 1.5%) found a broad optimum around 35% with 0.288 → 0.271; the adaptive estimate lands in the same region and removes the hand-tuned constant, which is the version measured in this paper. On the smaller log sample (35 records, 4 contexts over the threshold), the adaptive share gives 0.574 → 0.572 on that file’s own order-0-only baseline of 4590B — order-1 helps proportionally *more* on less training data, though the absolute win is small because the file is small.

Static per-record order-1 does not work; corpus-trained order-1 does. An earlier experiment tried order-1 as a *per-record* static table — built fresh from one record’s LZ-token stream, no shared training data, transmitted with the

record. Every variant (per-context tables, per-context and bucketed move-to-front) lost to plain per-record order-0 once table/escape transmission was counted: a single small record does not have enough repeated (context, symbol) pairs to amortize the cost of the table it transmits (“context dilution”). TokDict’s order-1 tables avoid this because they are trained *once*, across many records, and never transmitted — the same logic that justifies a static baked table over an online-adaptive one in [Section 4.6](#), applied one level up (across records sharing a schema rather than across positions within one record).

6.5 Throughput

All measurements are in-process, encoder/decoder constructed once and reused, pure Python throughout.

Table 4: In-process throughput. Target regime uses a TokDict trained on 184 structured-log records, measured on the 46 held-out records.

	Compress	Decompress
Target regime (TokDict)	11.5 rec/s	4899 rec/s
Bulk, prose	176 KB/s	186 KB/s
Bulk, Python code	185 KB/s	233 KB/s

Compression is far slower than decompression in the target regime specifically: the encoder builds and measures several candidate encodings per record ([Section 3](#)) and keeps the smallest, while the decoder only runs the one mode chosen. The asymmetry is much smaller in bulk mode, where the per-symbol rANS loop dominates both directions comparably. No throughput optimization was attempted ([Section 7](#)); read these as a correctness-first pure-Python baseline, not a tuned implementation.

6.6 Record-size crossover

To locate the crossover N^* of [Remark 4.5](#), we sweep record size over synthetic schema-homogeneous records (a fixed JSON-like key set with a randomized payload, 30 train / 10 held-out per size) and report per-record ratios for TokPress with and without a trained TokDict, and for zstd -19 with and without a matched trained dictionary.

A trained dictionary is most valuable precisely where generic per-record compression is worst: at 64 B, tokpress+dict is 0.424 versus 0.933 for dictionary-free zstd, and 0.309 for zstd’s own matched dictionary. The gap narrows steadily as records grow, and per-record adaptive zstd catches up around 1–2 KB — the crossover N^* for this schema. The middle band (128–512 B) is the awkward zone for *per-record* TokPress with these random-payload records:

Table 5: Per-record ratio by record size. The trained static model wins decisively at the smallest sizes and its advantage narrows as records grow.

Record size	tokpress	+TokDict	zstd	zstd+dict
64 B	1.061	0.424	0.933	0.309
128 B	1.129	1.003	0.923	0.488
256 B	1.257	1.257	0.816	0.618
512 B	1.181	1.181	0.761	0.642
1024 B	1.074	0.838	0.711	0.648
2048 B	1.019	0.799	0.685	0.660
4096 B	0.954	0.757	0.673	0.660

the dictionary cannot predict the randomized payload, and per-record table overhead dominates until the record is long enough to amortize it (which is exactly the regime Section 3’s batch mode, Section 6.2’s trained dictionary on real logs, avoid).

7 Conclusion

TokPress is a from-scratch, pure-Python, byte-exact lossless compressor that turns the tokenizer into the dictionary: it tokenizes with o200k.base, applies token-level LZ77, and entropy-codes with rANS, letting a trained TokDict and a batch mode make many small, similarly-structured records cheap to store. On the many-small-homogeneous-records regime it beats every dictionary-free baseline we tried and gets close to zstd’s matched dictionary; on bulk prose it now beats gzip on 4 of 6 corpora and zstd -19 outright on short prose. Every number is accompanied by a byte-exact round-trip check, and the code, the harness, and the tests are open source.

Two caveats gate how far these results generalize. First, TokPress still does not match zstd -19 with a trained dictionary — on the target JSON regime its best ratio is 1.33× zstd’s, down from 5–7× with no dictionary but not closed — and on a genuinely different schema a wrong-schema dictionary buys nothing over no dictionary at all. Second, the benefits of a trained dictionary depend on the schema and on how much training data you have; ratios should not be assumed to transfer to unmatched data without re-measuring.

The main open problems are a smarter dictionary-priming construction closer to zstd’s COVER/FastCover — a coverage-based selection (greedy by corpus frequent-token mass) is implemented and measured but does not beat head-concatenation on the paper-scale corpus, so diversity-aware selection remains open — an escape mechanism for the chunked-adaptive mode when a record’s vocabulary exceeds M , and a vendored, fully reproducible corpus setup. The cross-schema experiment, the implementation constants, the compression-and-security discussion, and the reproducibility notes are collected in the appendix.

A Theorems and Proofs

All formal statements referenced from the main text are collected here.

Lemma A.1 (Coding identity). *For any model q on X and true distribution p ,*

$$\mathbb{E}_p[-\log_2 q(X)] = H(p) + D(p \parallel q).$$

Proof. Decompose $\log_2(1/q) = \log_2(1/p) + \log_2(p/q)$ and apply definitions. $\square$

Theorem A.2 (Invertibility and entropy invariance). *For all $x \in \mathcal{B}^*$, $\kappa(T(x)) = x$. Hence T is injective and $H(T(X)) = H(X)$ for any source.*

Proof. Byte fallback guarantees greedy matching never stalls, so $\kappa(T(x)) = x$. If $T(x) = T(x')$ then $x = \kappa(T(x)) = \kappa(T(x')) = x'$. An injective map preserves the entropy of a discrete random variable. $\square$

Corollary A.3 (No free lunch). *Tokenization cannot reduce the information content of a message. Every compression gain must come from modeling, never from the transform.*

Theorem A.4 (Adjacent-token mutual information).

$$R_0^T - R_1^T = \frac{1}{\ell} I(T_0; T_1) \geq 0.$$

Proof. $H(T_1) - H(T_1 | T_0) = I(T_0; T_1)$; divide by ℓ . $\square$

Lemma A.5 (Entropy growth of a Zipf vocabulary). *For $p_i \propto i^{-\alpha}$, $i \leq K$,*

$$H(K) \sim \begin{cases} \log_2 K, & \alpha < 1, \\ \frac{1}{2} \log_2 K + \log_2(\ln K), & \alpha = 1, \\ H_\infty < \infty, & \alpha > 1. \end{cases}$$

Proof sketch. Euler–Maclaurin. For $\alpha = 1$, $Z = \ln K + O(1)$ and $\sum_{i \leq K} (\ln i)/i = \frac{1}{2}(\ln K)^2 + O(1)$. For $\alpha < 1$, $Z \sim K^{1-\alpha}/(1-\alpha)$ and $\sum i^{-\alpha} \ln i \sim Z \ln K$. For $\alpha > 1$ both sums converge. $\square$

Proposition A.6 (Monotone rate; no interior optimum). *With $r(K) = a + \beta \ln K$, $H(K) = \frac{1}{2} \log_2 K + c$, and constant model divergence δ , the per-byte rate is*

$$b(K) = \frac{\frac{1}{2\ln 2} \ln K + c + \delta}{a + \beta \ln K}, \quad \frac{db}{d \ln K} = \frac{\frac{a}{2\ln 2} - \beta(c + \delta)}{(a + \beta \ln K)^2},$$

whose numerator is independent of K . Hence b is monotone and there is no interior argmin.

Theorem A.7 (Finite-length LZ redundancy). *For a stationary ergodic source of entropy rate h bits/byte, the sliding-window LZ77 rate over N bytes satisfies $b_{LZ}(N) = h + \Theta(\log \log N / \log N)$ [18, 12].*

Theorem A.8 (Correctness). *The rANS step is a bijection; given the exact frequency model, the compressed stream uniquely decodes.*

Proof. $b = (s \bmod f_i) + c_i \in [c_i, c_i + f_i)$ determines i uniquely. Then $b - c_i = s \bmod f_i$ and $\lfloor s'/M \rfloor = \lfloor s/f_i \rfloor$, so the decoder returns $f_i \lfloor s/f_i \rfloor + (s \bmod f_i) = s$. Renormalization streams are LIFO-inverse by construction. $\square$

Theorem A.9 (rANS optimality). *The expected per-symbol length under model q against true p is*

$$\mathbb{E}[\ell] = \sum_i p_i \log_2 \frac{M}{f_i} = H(p) + D(p \parallel q).$$

Proof. Each encode step contributes $\log_2(M/f_i) = -\log_2 q_i$ bits, the state being confined to a bounded interval; apply [Lemma A.1](#). $\square$

Theorem A.10 (Master rate equation). *With $\tilde{p}$ the distribution of the LZ-transformed token stream,*

$$\frac{|c|}{|x|} = \frac{1}{\tilde{\ell}} \left[H(\tilde{p}) + D(\tilde{p} \parallel \tilde{q}) + \varepsilon_{\text{quant}} + \varepsilon_{\text{flush}} + \varepsilon_{\text{header}} + \varepsilon_{\text{LZ}} \right],$$

where $\varepsilon_{\text{quant}} \rightarrow 0$ as $M \rightarrow \infty$, $\varepsilon_{\text{flush}} = 32W/M_b$, $\varepsilon_{\text{header}}$ is fixed per-record overhead, and $\varepsilon_{\text{LZ}} \geq 0$ is parse inefficiency.

Proof. Apply [Theorem A.9](#) to the transformed stream, divide by $|x| = m\tilde{\ell}$, and collect non-KL terms. $\square$

Theorem A.11 (No semantic security). *Encode $_{\Theta}$ is deterministic, hence trivially distinguishable: equal ciphertexts imply equal plaintexts.*

Theorem A.12 (No integrity). *A single flipped bit can corrupt the decoded message without bound from the point of error, undetectably: the rANS state machine is serial, a corrupted state word propagates through renormalization into every subsequent symbol, and there are no sync markers or checksums.*

Theorem A.13 (Code-length leak). *By [Theorem A.9](#), $|c(x)| = \sum_i n_i(x) \log_2(M/f_i) + O(1)$ with n_i the token counts of x . Given N known-plaintext pairs, an adversary obtains an overdetermined bilinear system in the unknown frequencies $\{f_i\}$ and the unknown segmentation, solvable by standard estimation whenever the tokenizer is constrained by the domain. The frequencies are consistently estimable as $N \rightarrow \infty$.*

Corollary A.14. *The construction provides obfuscation rather than cryptographic confidentiality. Known plaintext exposes information about the tokenizer and frequency model, while chosen single-byte inputs directly reveal the corresponding byte IDs.*

B Cross-Schema Generalization

This experiment asks whether a trained TokDict survives a different *kind* of record, not just a different JSON shape.

Method. A TokDict is trained on 184 JSON log records (the same training split as [Section 6.2](#)) and evaluated, unchanged, on 22 held-out Python code snippets (function/class bodies, 100–2000 B each, split from an 108-snippet pool at 80/20). We compare against a second TokDict trained on the matching 86 code snippets, and against zstd -19 with no dictionary, a dictionary trained on the mismatched JSON schema, and a dictionary trained on the matched code schema.

Table 6: Cross-schema test: JSON-trained dictionary applied to code snippets (14637 B total, 22 held-out records). All TokPress rows round-tripped losslessly.

Method	Ratio
TokPress+TokDict, JSON-trained (wrong schema)	0.489
zstd -19, no dictionary	0.485
zstd -19, dict. trained on wrong (JSON) schema	0.486
TokPress+TokDict, code-trained (matched)	0.484
zstd -19, dict. trained on matched (code) schema	0.379

Result. The wrong-schema TokDict (0.489) is statistically indistinguishable from zstd with no dictionary (0.485): a dictionary trained on the wrong kind of record buys nothing here — a stronger result than a same-domain schema shift, because JSON log records and Python code snippets share almost no token-level structure to transfer. A TokDict trained on the *matched* schema (0.484) is barely better than none, and zstd’s matched dictionary (0.379) stays well ahead of both. This fits the coding model ([Theorem A.9](#)): when the trained model does not match the data, the mismatch term dominates; and 86 training records are simply too few (vs. 184 in [Section 6.2](#)) for the order-1 tables to find much reusable structure even on a matched schema. The results should not be read as applying to any input regardless of schema: a TokDict is a domain-specific artifact, and its benefit shows up only when training data resembles the data it will compress.

C Concrete Instantiation

Remark C.1 (Why a 64-bit coder state). This is pure Python (arbitrary-precision integers), so no register width forces a 32-bit state; the earlier $M = 2^{12}$ configuration used 32 bits because $L \cdot 2^{16} = 2^{32}$ exactly at that precision, leaving no headroom to raise M . Raising M to 2^{16} at a fixed 32-bit mask was checked by hand first and found to overflow 32 bits in edge cases (maximum frequency/cumulative-frequency combinations), so the state was widened to 64 bits, keeping $L = 16M$ (the same ratio the 2^{12} configuration used). The headroom was verified analytically (worst-case post-update state bounded under 2^{36}) and empirically (a synthetic worst-case-skew stress test, alongside the full test suite and every [Section 6](#) corpus’s round-trip check).

Remark C.2 (Alphabet sparsity). The rANS alphabet is bounded to $M-1$ real symbols plus one escape symbol per table, but the number of symbols that actually occur in any one record or context is usually far smaller. Every sparse/split/adaptive mode transmits only the list of symbol ids that are active (sorted, delta-encoded, varint-packed), never a dense array over the full alphabet — paying $\log_2 M$ bits for every possible symbol id would dominate a per-record table for any text with more than a few hundred distinct tokens.

Table 7: Constants as implemented.

Quantity	Value	Note
Byte alphabet $\mathcal{B}$	256	byte fallback
Tokenizer vocabulary $\mathcal{V}$	$\approx 2 \times 10^5$	full o200k_base, unfiltered
rANS precision M	2^{16}	frequency total
rANS coder state	64-bit	see remark below
rANS renorm bound L	$16M = 2^{20}$	renormalization
rANS word width	16 bits	renorm quantum
Token ID width (raw mode)	≈ 18 bits	$\lceil \log_2 \mathcal{V} \rceil$, dynamic
Match flag	$ \mathcal{V} $	token-LZ marker, dynamic
Max match distance D	2^{15}	token-LZ
Max match length	255	token-LZ
Match threshold	3	Remark 4.4
LZ priming buffer	≤ 8192 tok	per TokDict, trainable
Order-1 contexts	top 64	≥ 20 transitions
Order-0 escape share	1.5%	TokDict
Order-1 escape share	PPMC-adaptive	TokDict, count-based (Section 6.4)
Adaptive chunk size	≥ 256	scales with $n \cdot k$, Section 7

D Discussion: Compression and Security

As a proposed extension, we consider whether a tokenizer trained on private data could double as a secret key. The construction is not implemented in TokPress, makes no security claim for the released codec, and is *not* a contribution of this paper; the analysis is recorded in [Appendix E](#) because it shapes what a trained dictionary must *not* be used for. The short version: a private trained model provides obfuscation, not confidentiality — the encoder is deterministic, a flipped bit corrupts output undetectably, and code lengths leak token counts — so compression must be followed by authenticated encryption whenever confidentiality is required. Details, formal statements, and the secure-composition recommendation are in [Appendix E](#) ([Theorems A.11](#) to [A.13](#) and [Corollary A.14](#)).

E Keyed-Model Analysis (non-contributory)

This appendix records, for completeness, the analysis referenced in the main-text discussion ([Appendix D](#)). It is not a contribution of this paper and makes no security claim for the released codec.

Let a keyed codec be $\Pi = (\text{Train}, \text{Encode}, \text{Decode})$, where $\text{Train}(C, \theta) \mapsto \Theta = (\mathcal{V}_\Theta, q_\Theta)$ from a private corpus C and secret seed θ . Correctness ($\text{Decode}_\Theta \circ \text{Encode}_\Theta = \text{id}$) follows from [Theorem A.2](#) and [Theorem A.8](#). Under Kerckhoffs’ principle the algorithm is public and security rests entirely on secrecy of Θ .

Fact E.1. If $\mathcal{V}_\Theta$ is public, as with a released tokenizer, it provides no secrecy. If $\mathcal{V}_\Theta$ is private, the resulting construction is a codebook cipher in the substitution-cipher family.

Remark E.2 (The compression–security tension). $D(p \parallel q_\Theta)$

affects both coding rate ([Theorem A.9](#)) and model recovery ([Theorem A.13](#)). A model close to p improves compression, but observations from p also give more direct evidence about that model. Better training alone therefore does not turn the codec into a secure encryption scheme.

Remark E.3 (Worst-case hardness is not protection). Dictionary learning is NP-hard in the worst case [7], but that does not establish practical security. Structured text may admit effective greedy or EM-style recovery despite worst-case complexity.

Secure composition. When confidentiality is required, compression must be followed by authenticated encryption, $c = \text{AEAD.Enc}(k, \text{Encode}_\Theta(x))$, which addresses [Theorem A.11](#), [Theorem A.12](#) and length-oracle concerns at once. In that composition, security comes from the authenticated cipher, not from the tokenizer or entropy model.

References

- [1] T. M. Cover and J. A. Thomas. *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [2] G. Delétang, A. Ruoss, et al. Language Modeling Is Compression. arXiv:2309.10668, 2023.
- [3] J. Duda. Asymmetric numeral systems. arXiv:0902.0271, 2009.
- [4] G. Kelly. FemtoZip: a compression library for short documents. Open-source project.
- [5] P. Gage. A New Algorithm for Data Compression. *The C Users Journal*, 12(2), 1994.
- [6] A. Grattafiori et al. The Llama 3 Herd of Models. arXiv:2407.21783, 2024.
- [7] R. Gribonval, R. Jenatton, and F. Bach. Sparse and spurious: dictionary learning with noise and outliers. *IEEE Trans. Inf. Theory*, 61(11), 2015.
- [8] H. S. Heaps. *Information Retrieval: Computational and Theoretical Aspects*. Academic Press, 1978.
- [9] R. Krichevsky and V. Trofimov. The performance of universal encoding. *IEEE Trans. Inf. Theory*, 27(2):199–207, 1981.
- [10] T. Kudo. Subword Regularization. *Proc. ACL*, arXiv:1804.10959, 2018.
- [11] T. Kudo and J. Richardson. SentencePiece. *EMNLP (demo)*, arXiv:1808.06226, 2018.
- [12] G. Louchard and W. Szpankowski. On the average redundancy rate of the Lempel–Ziv code. *IEEE Trans. Inf. Theory*, 43(1):2–8, 1997.
- [13] B. Tang, S. Yang, Z. Shen, W. Zhang, X. Lin, and Z. Tian. LogLite: Lightweight Plug-and-Play Streaming Log Compression. *Proc. VLDB*, 18(11), 2025. arXiv:2507.10337.
- [14] OpenAI. tiktoken (o200k_base). GitHub repository, 2024.
- [15] A. Radford, J. Wu, et al. Language Models are Unsupervised Multitask Learners. OpenAI, 2019.
- [16] Y. Collet and M. Kucherawy. Zstandard Compression and the application/zstd Media Type. RFC 8478, 2018.

- [17] R. Sennrich, B. Haddow, and A. Birch. Neural Machine Translation of Rare Words with Subword Units. *Proc. ACL*, arXiv:1508.07909, 2016.
- [18] A. D. Wyner and J. Ziv. The sliding-window Lempel–Ziv algorithm is asymptotically optimal. *Proc. IEEE*, 82(6):872–877, 1994.
- [19] G. K. Zipf. *Human Behavior and the Principle of Least Effort*. Addison-Wesley, 1949.
- [20] J. Ziv and A. Lempel. Compression of individual sequences via variable-rate coding. *IEEE Trans. Inf. Theory*, 24(5):530–536, 1978.